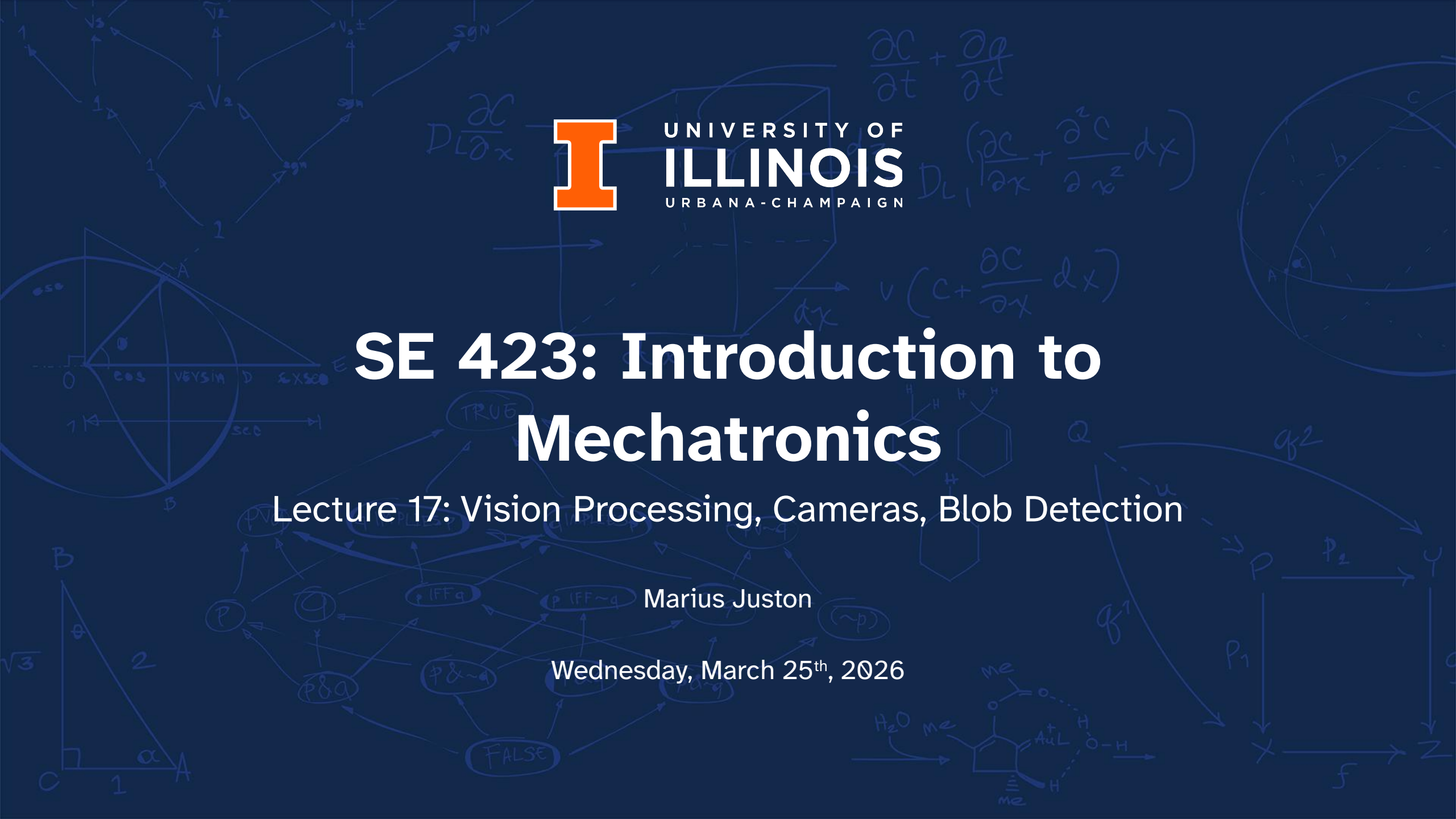


UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

# SE 423: Introduction to Mechatronics

## Lecture 17: Vision Processing, Cameras, Blob Detection

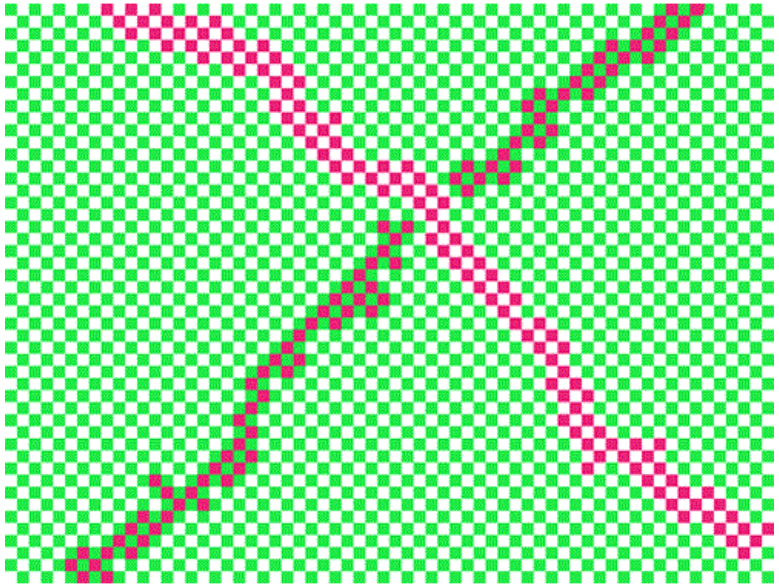
Marius Juston

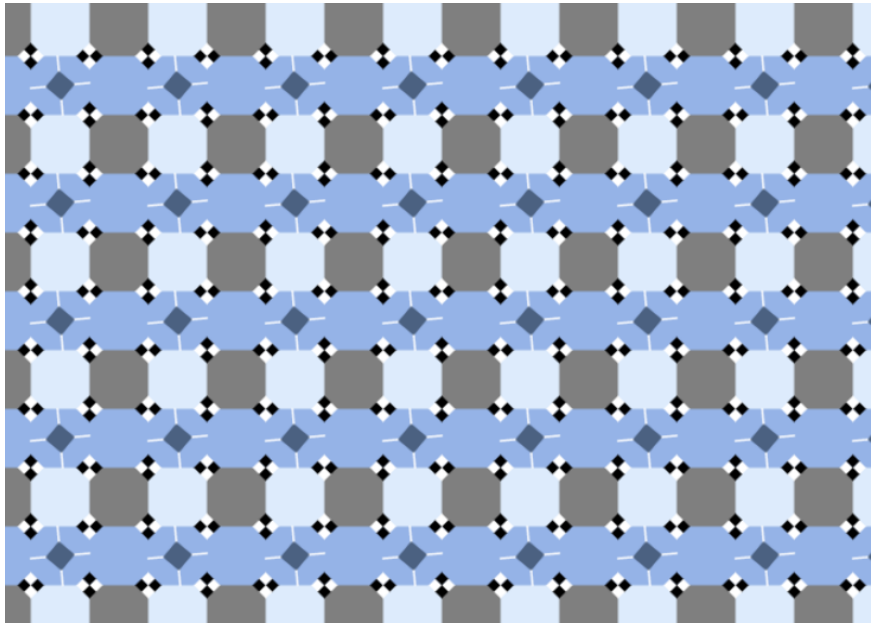
Wednesday, March 25th, 2026

# Review – 5-minute partner discussion

Talk to the person next to you and answer these questions.

- What does RGB stand for?
- How many colors are there? Are the lines straight?





check out https://michaelbach.de/ot/ for a bunch of optical illusions!

# Reminders

**Office Hours:**

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

**Lab:** Starting Lab 6 this week. This a 3-week lab!

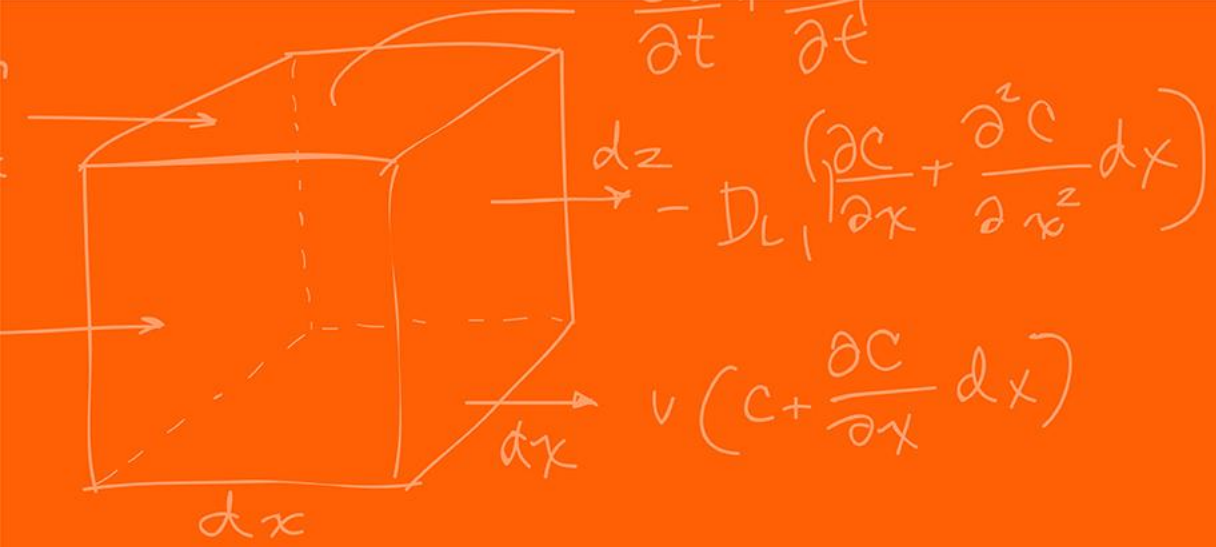
- Don’t forget to have the Lab LabVIEW application implemented before lab.

**Homeworks:**

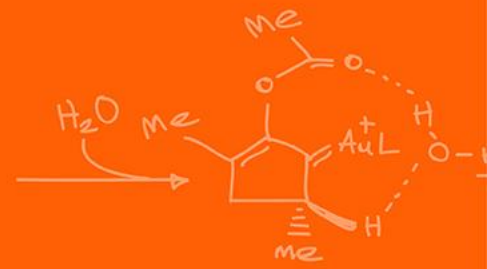
- Check-offs for homework 4 Ex 2 and 3 are due **March 31, 5PM (The end of office hours)**
- Check-offs for homework 4 Ex 4 are due **April 7, 5PM (The end of office hours)**
- Remainder of homework 4 due on Gradescope on **April 8, 9AM**

# Questions? (1/2)

Homework, Lab, LabVIEW?

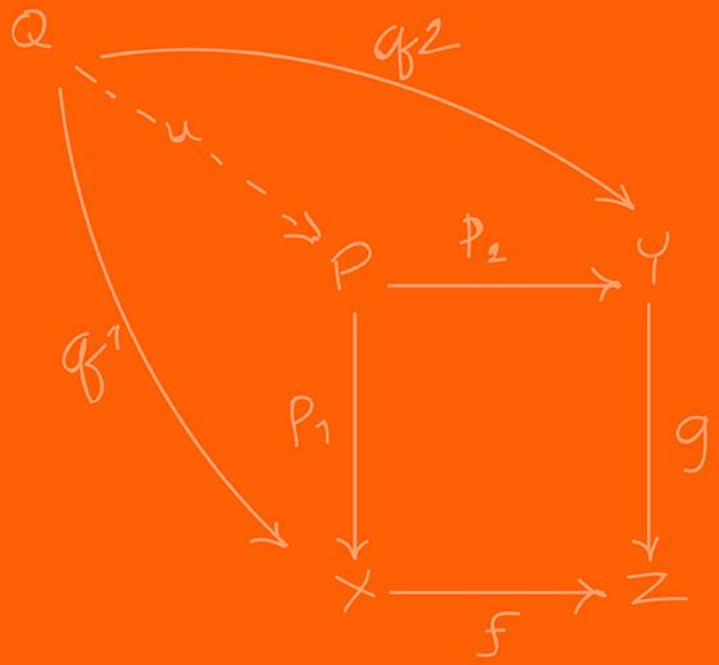








# Computer Vision



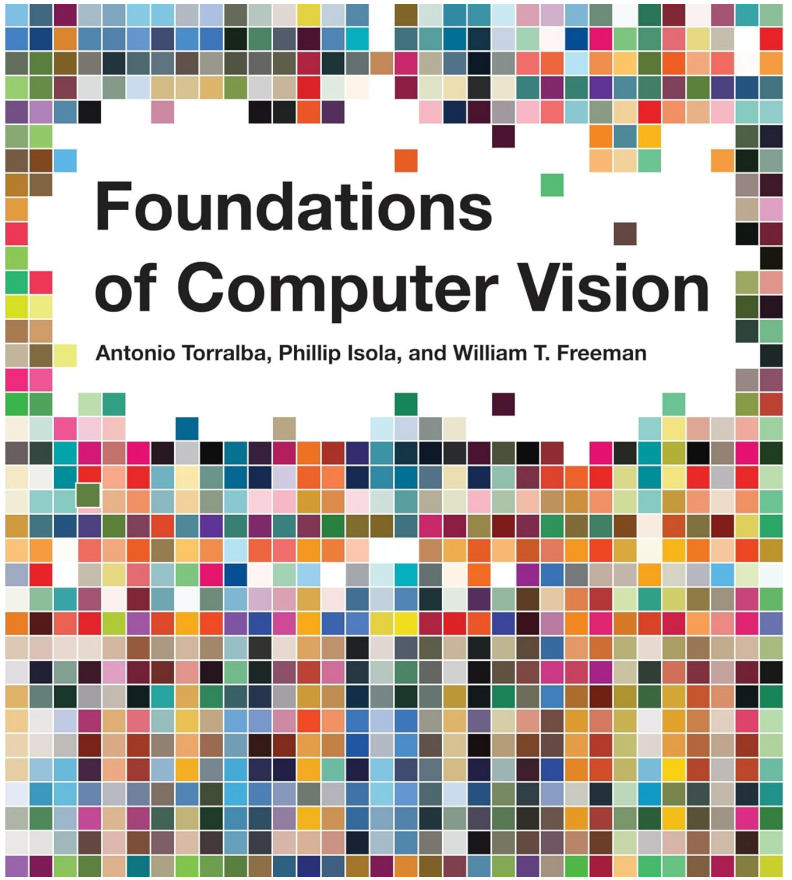
# Preface

I would **HIGHLY** recommend you looking at:

- https://visionbook.mit.edu/

- The webpage is a textbook with all the math, background and information for computer vision.
- We will only be doing a tiny portion of this in this lecture (there are whole grad classes for computer vision).
- It is containerized very nicely, has lots of pictures, diagrams. Each section takes 5 minutes to read.

- (also, for more CMOS details look at https://isl.stanford.edu/~abbas/ee392b/lect04.pdf )



# Camera Vision (1/33)

Currently we have the LiDAR working and returning points; however, what kind of data are missing from just the LiDAR?

Color, complex shapes, human vision like data

Vision processing allows a robot to extract rich, high-density data from its environment.

Instead of an array of 228 distances (like our LiDAR), a camera gives us millions of data points per frame (RGB per pixel).

What is the primary drawback of having millions of data points per frame in a real-time mechatronic system?

Computational latency; processing takes time, more memory / bandwidth is required to handle processing of the data!

# Camera Vision (2/33)

Is a standard camera an active or a passive sensor?

Passive.

It does not emit energy; it captures ambient light bouncing off objects.

Because it is passive, it is highly vulnerable to environmental changes.

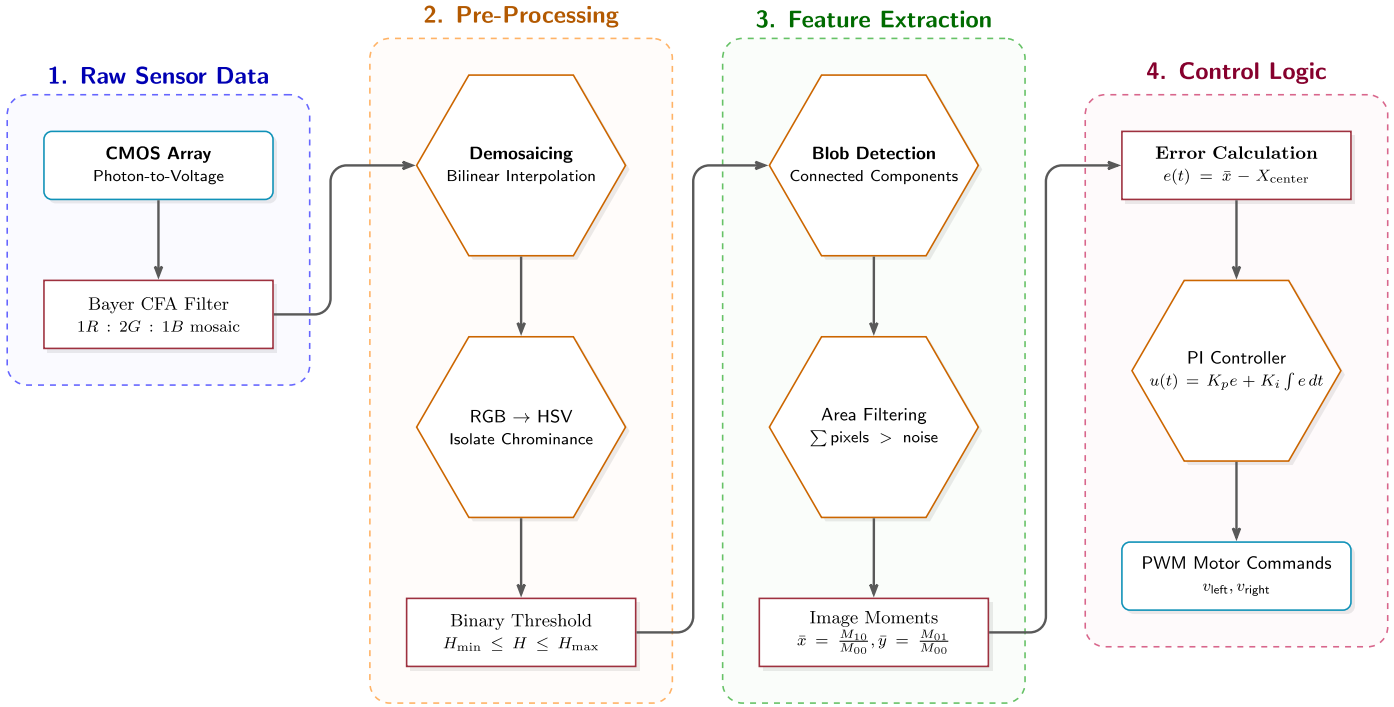
If the sun goes behind a cloud what happens?

The pixel color changes, shadows appears, etc.

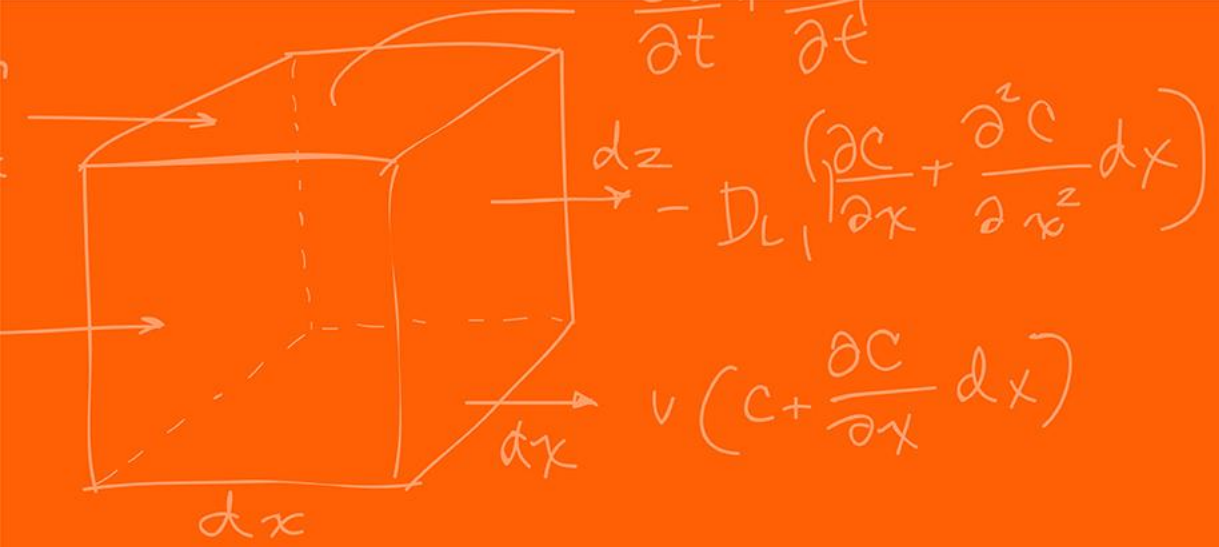
How do we build robust code around this?

# Camera Vision (3/33)

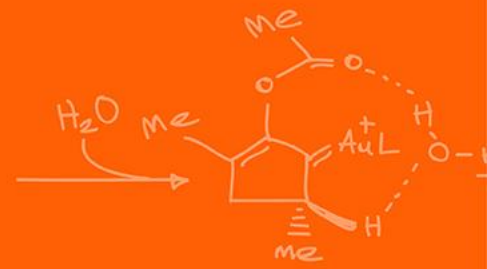
The main pipeline that we will follow in this class is the following:




Step 1: The CMOS sensor captures photons and converts them to digital arrays.
Step 2: The processor filters the image (e.g., finding specific colors).
Step 3: The algorithm calculates the coordinates of the target (Centroid).
Step 4: The Cascade Control loop turns the coordinates into wheel velocities.

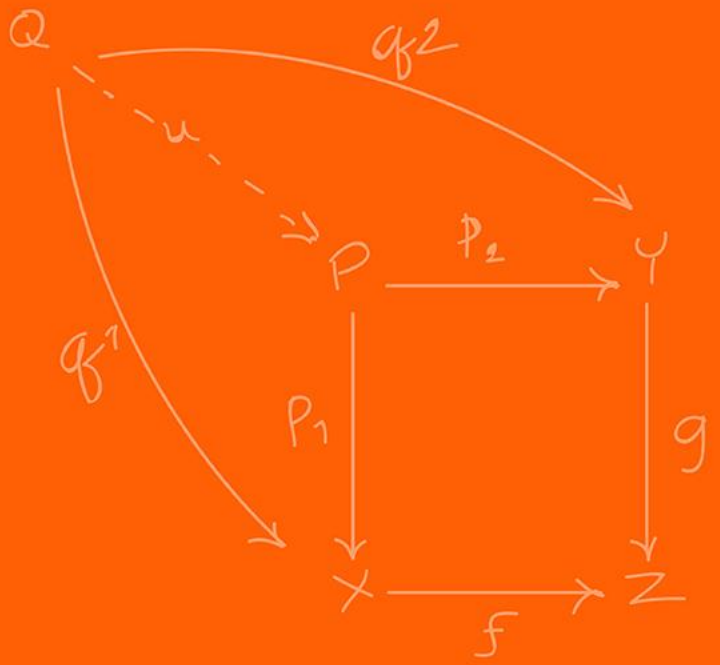




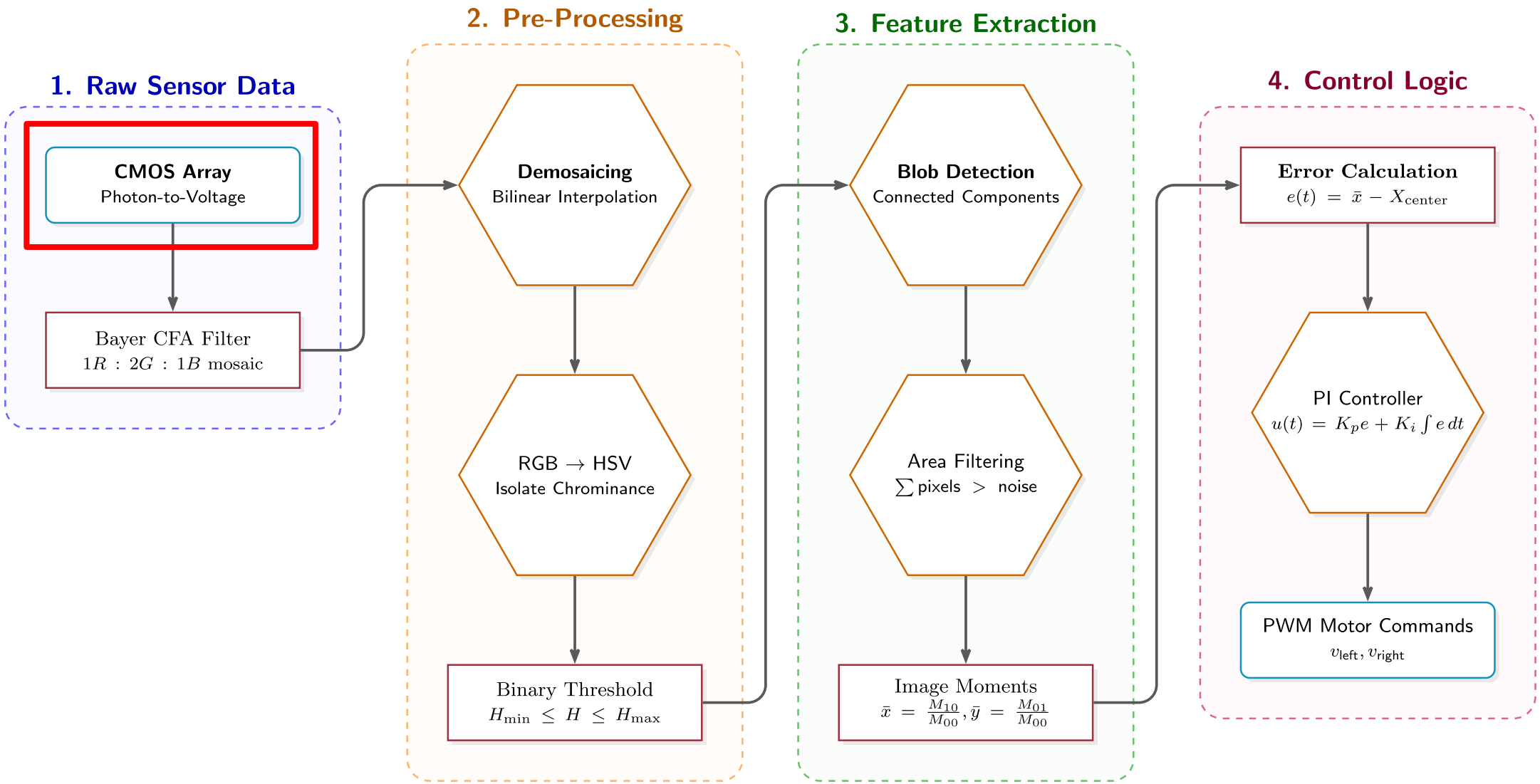




# CMOS



# Camera Vision (4/33)



# Camera Vision (5/33)

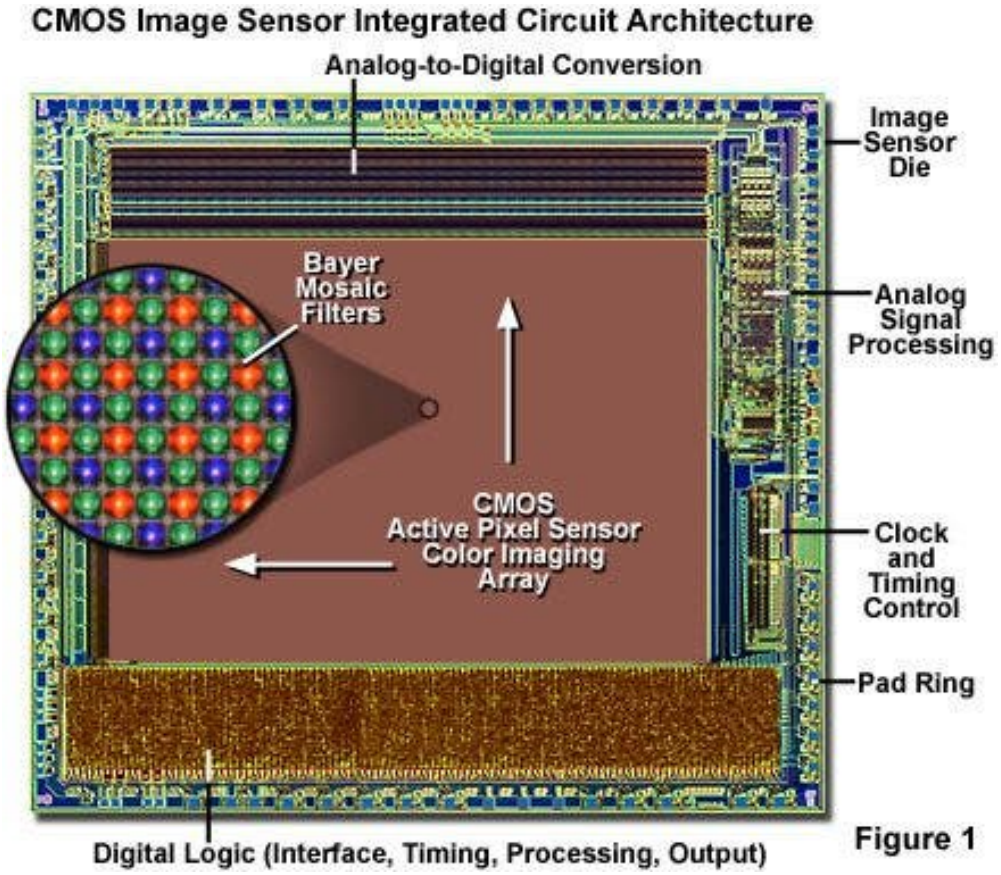
Most modern robotics vision relies on CMOS (Complementary Metal-Oxide-Semiconductor) sensors.

A CMOS sensor contains a massive 2D grid of microscopic photodiodes.

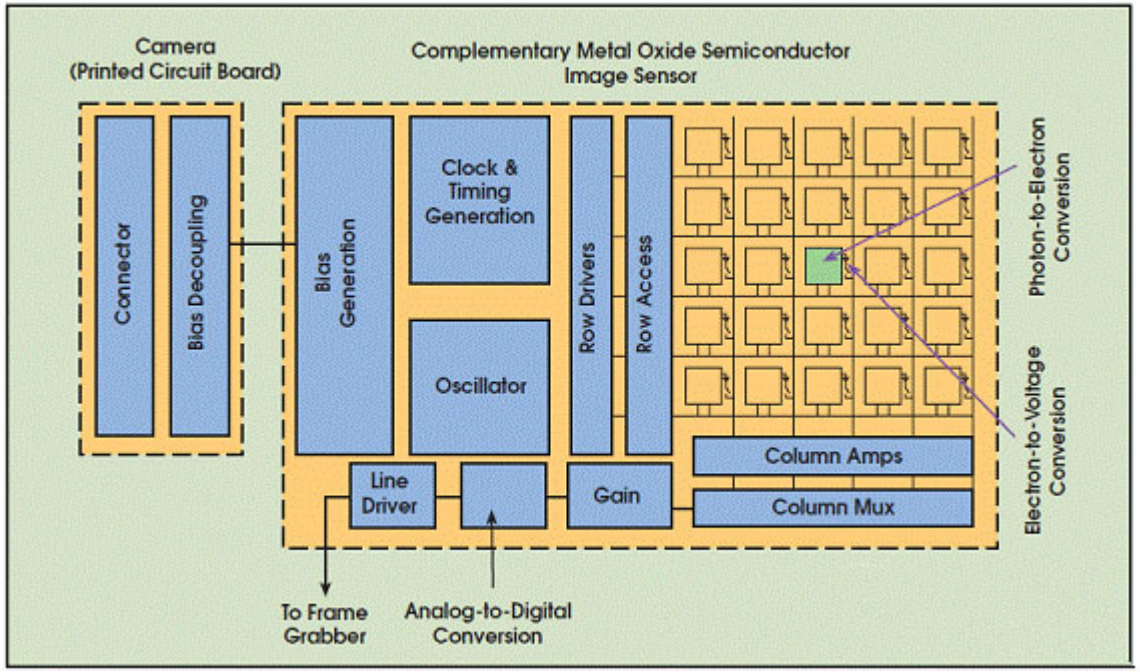
When photons strike a photodiode, they excite electrons via the photoelectric effect, creating a measurable voltage.

This continuous voltage is sampled and converted into a discrete digital value (usually 0-255 for an 8-bit ADC).

# Camera Vision (6/33)

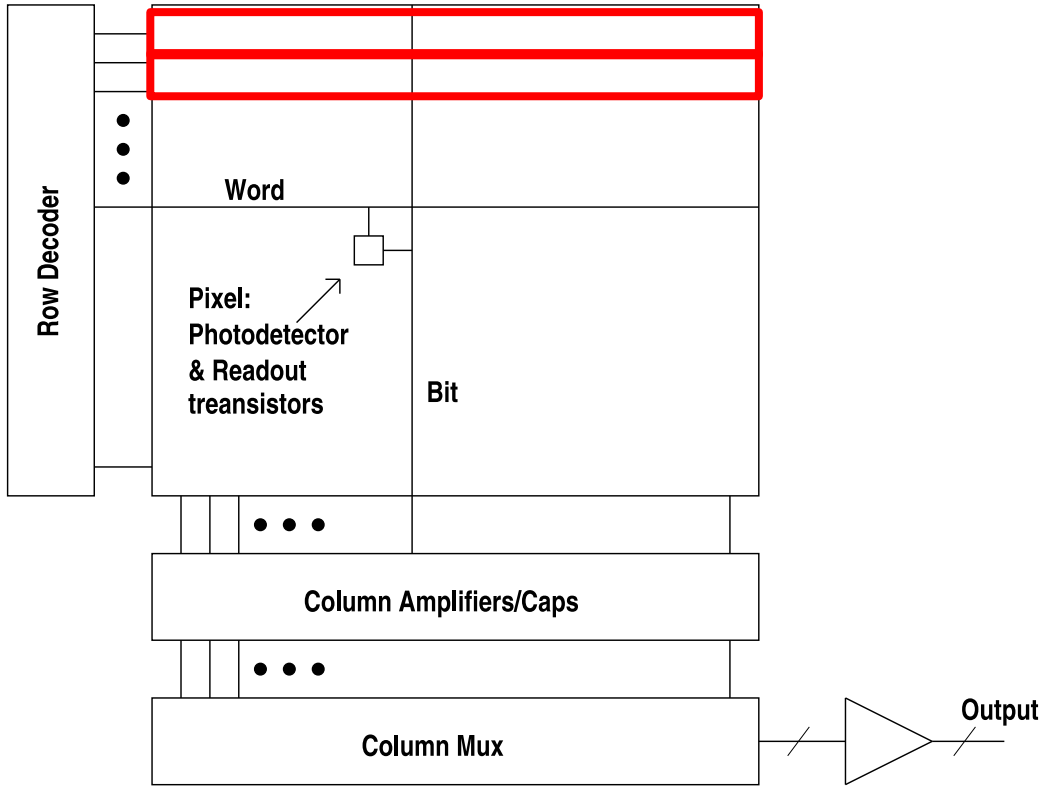



https://evidentscientific.com/en/microscope-resource/knowledge-hub/digital-imaging/cmosimagesensors




https://evidentscientific.com/en/microscope-resource/knowledge-hub/digital-imaging/cmosimagesensors

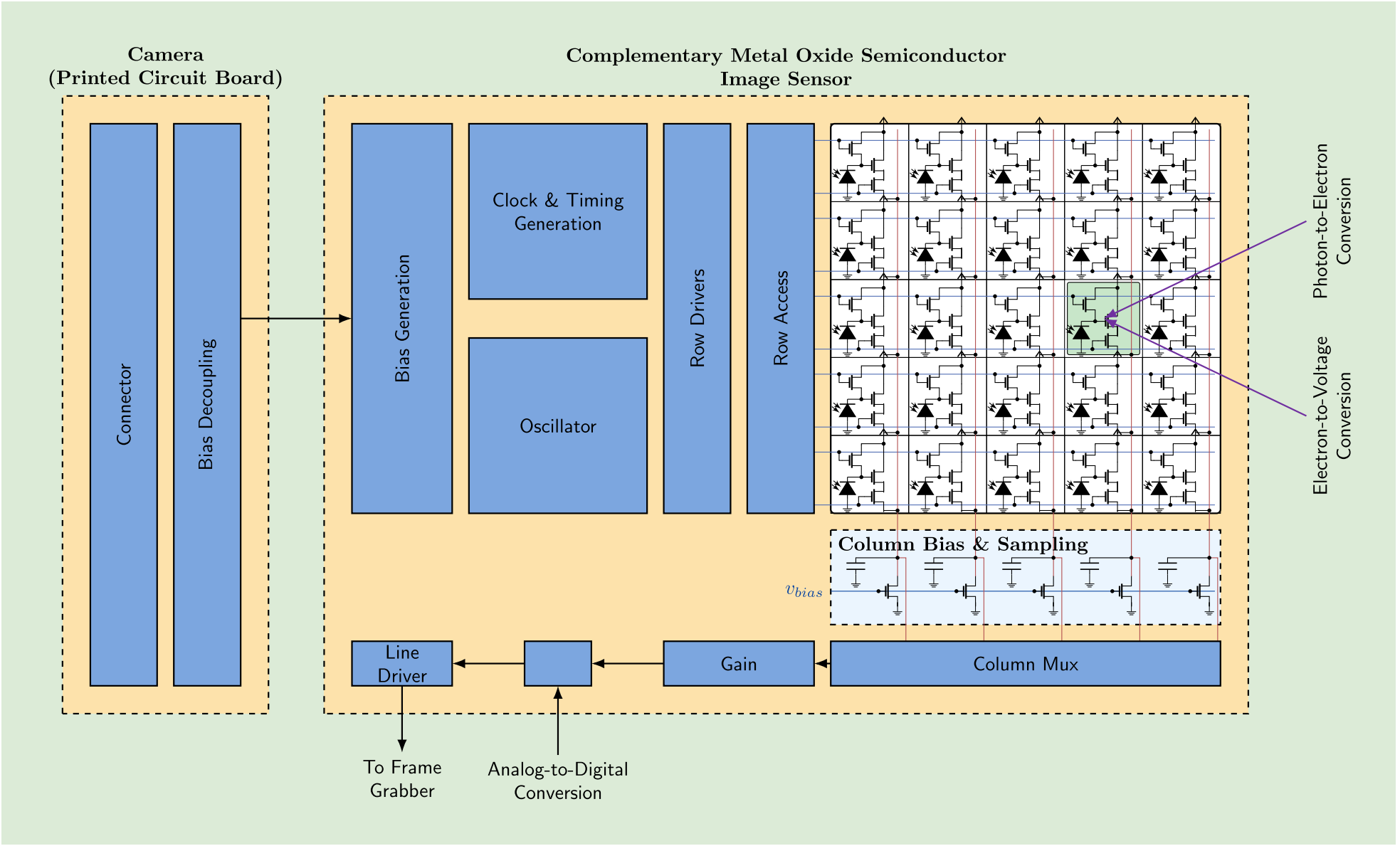
# Camera Vision (7/33)

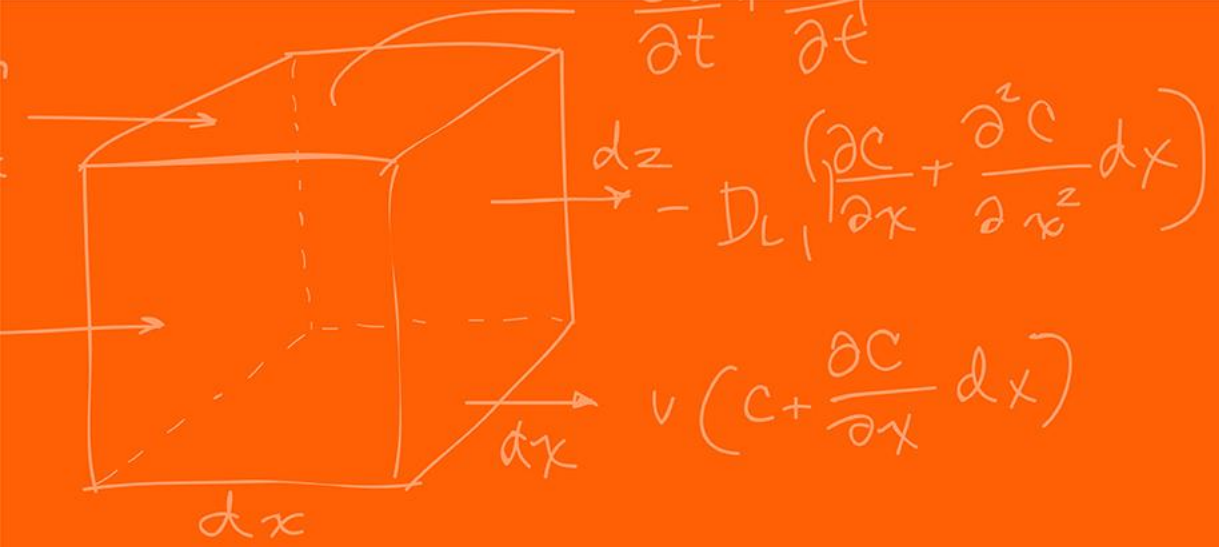



https://isl.stanford.edu/~abbas/ee392b/lect04.pdf

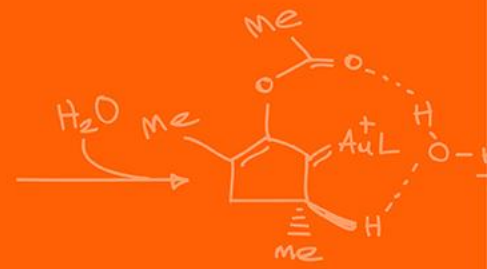
- Readout performed by transferring one row at a time to the column storage capacitors, then reading out the row, one (or more) pixel at a time, using the column decoder and multiplexer
- In many CMOS image sensor architectures, row integration times are *staggerred* by the row / column readout time (scrolling shutter)

# Camera Vision (8/33)



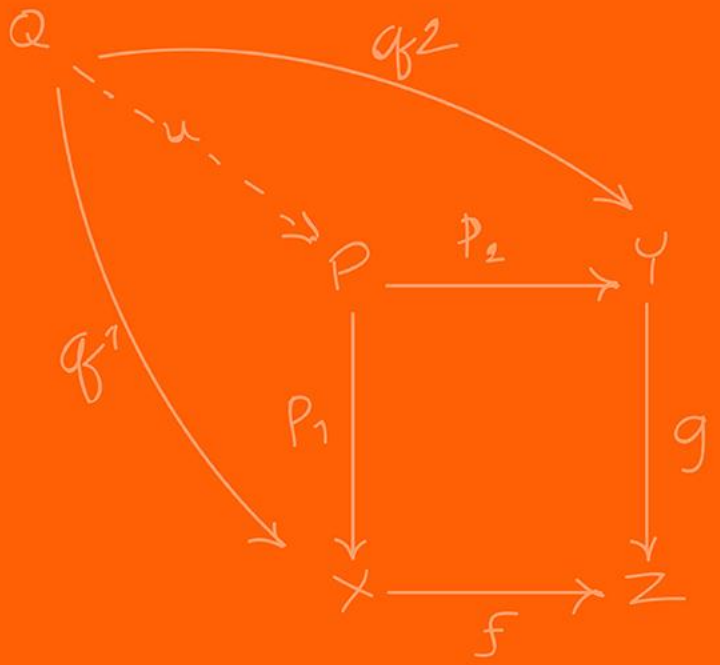




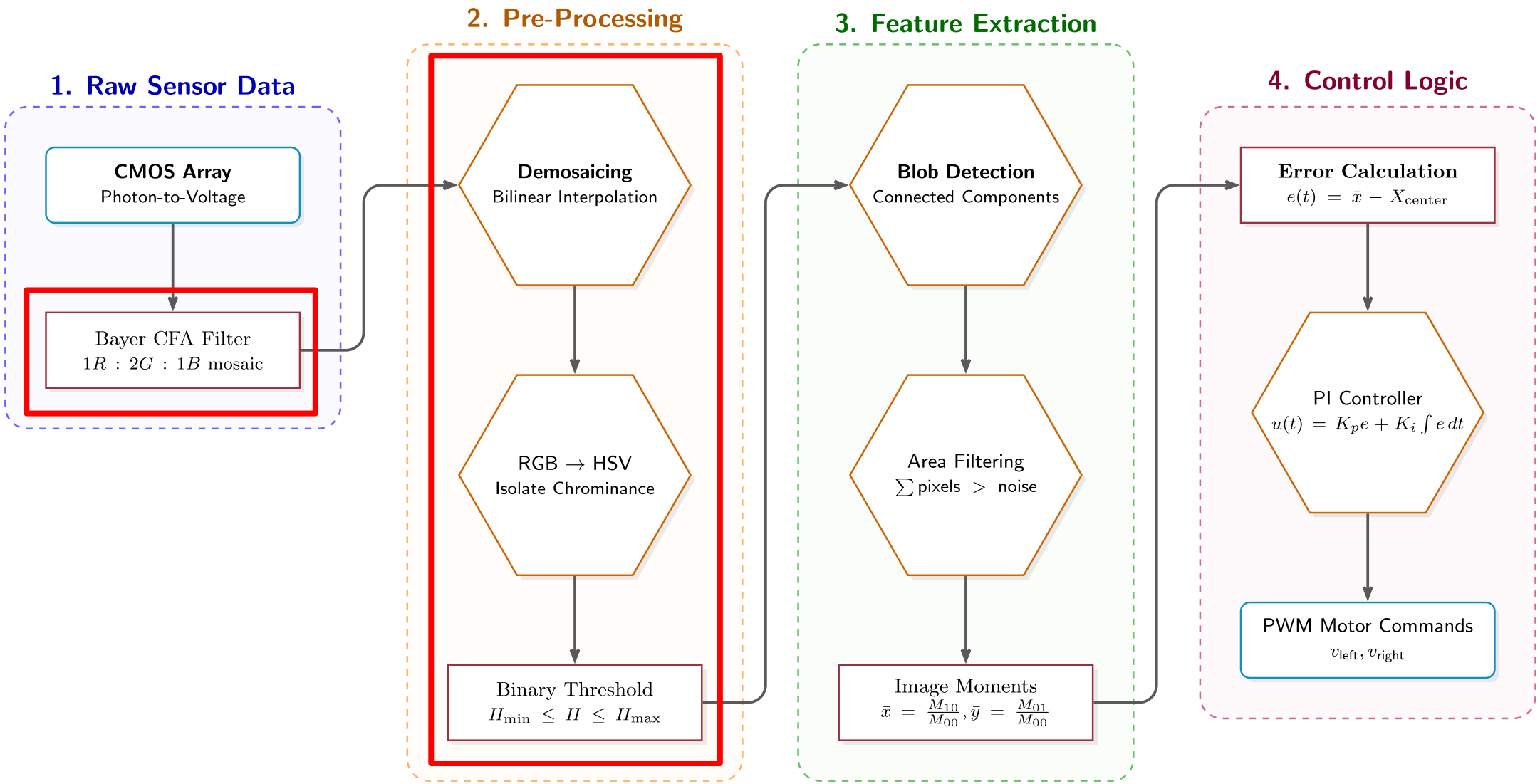


# Color





# Camera Vision (9/33)



# Camera Vision (10/33)

The problem with photodiodes is that they only measure the intensity of light (how many photons hit them).

What kind of problem that cause?

They are inherently colorblind; they only produce a grayscale image!

To see a red ball, we need a way to filter out all other wavelengths of light before they hit the photodiode.

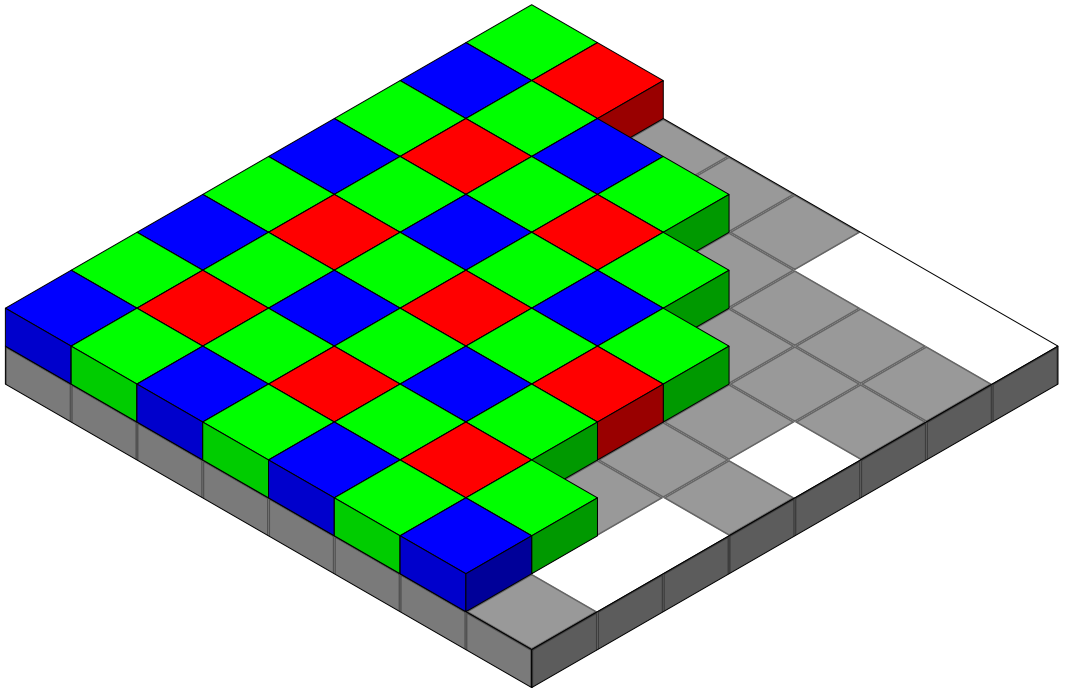
How do we make a single sensor array see Red, Green, and Blue simultaneously?

# Camera Vision (11/33)

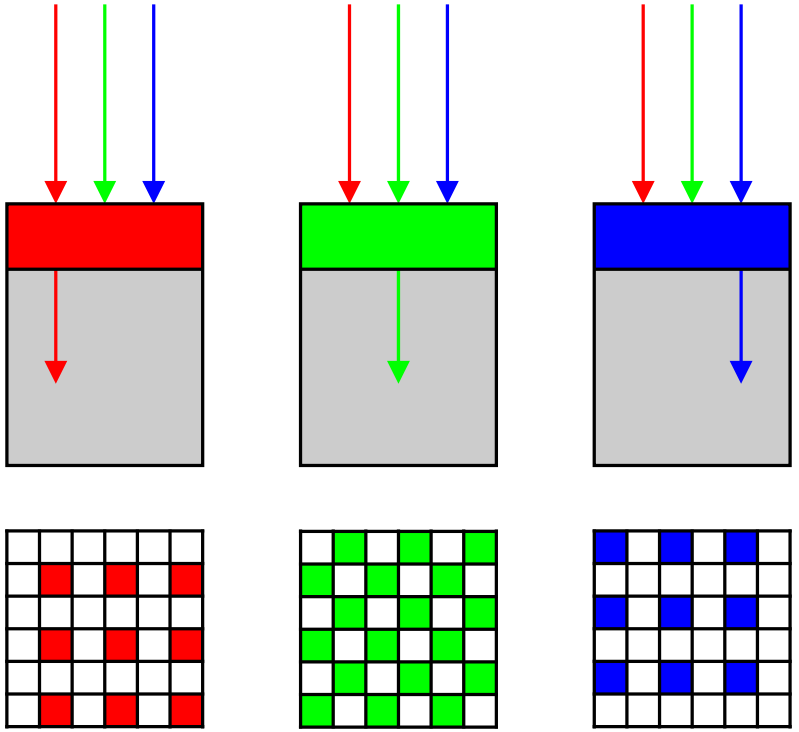
To achieve color vision, a microscopic color filter is placed over each individual photodiode.

This is known as a Color Filter Array (CFA), the most common being the Bayer Filter.

It assigns each pixel to capture only Red, only Green, or only Blue light.



https://en.wikipedia.org/wiki/Bayer_filter



# Camera Vision (12/33)

The Bayer format arranges filters in a repeating $2 \times 2$ grid.

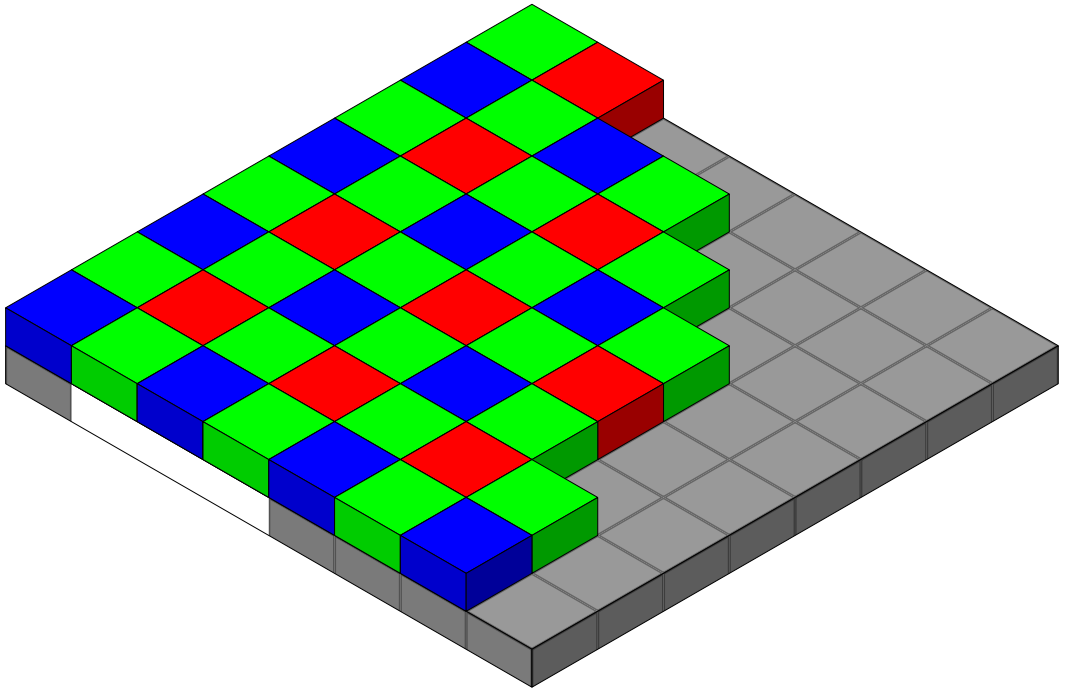
What is a problem with filtering color?

We lose information! This is why most scientific instruments (on Mars for example) are in black and white to ensure capturing all the intensities!

Notice anything strange about this ratio?

1 Red : 2 Green : 1 Blue

Why are there twice as many Green pixels as Red or Blue?



https://en.wikipedia.org/wiki/Bayer_filter

# Camera Vision (13/33)

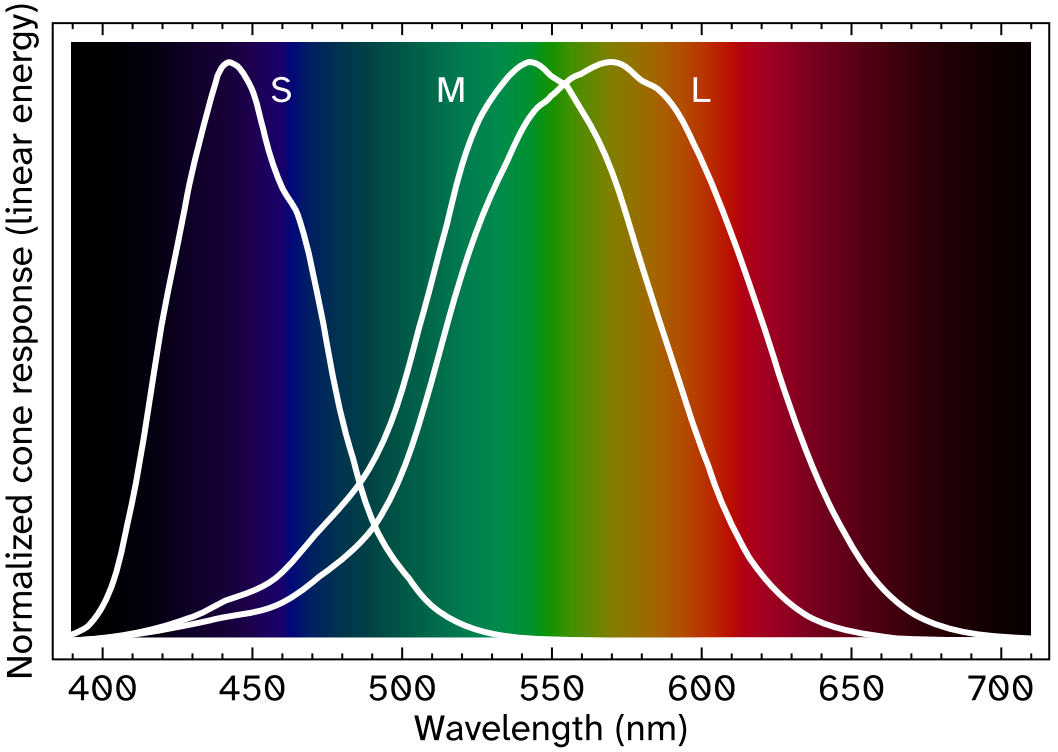
We use twice as many green filters because the human eye (specifically our cone cells) is most sensitive to green wavelengths (~550 nm).

By over-sampling green, the camera captures luminance (brightness) detail that matches human perception.

However, our raw array is now a mosaic of single colors.

Pixel (0,0) only knows Red.

How do we get RGB values for every pixel?




https://en.wikipedia.org/wiki/Color_vision

# Camera Vision (14/33)

**Demosaicing** is the mathematical process of guessing the missing color values at each pixel location.

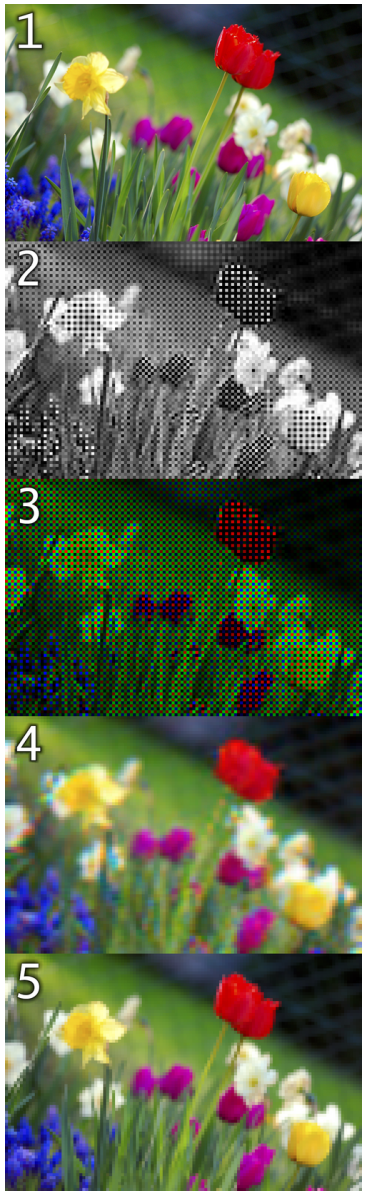
We use surrounding pixels to calculate an average. To find the missing Green value at a Red pixel:

$$R_{i,j}: G_{i,j} = \frac{1}{4}\left(G_{i-1,j} + G_{i+1,j} + G_{i,j-1} + G_{i,j+1}\right)$$

The camera's internal DSP (Digital Signal Processor) usually does this before sending data to the microcontroller.

This simple approach works well in areas with constant color or smooth gradients, but it can cause artifacts such as color bleeding in areas where there are abrupt changes in color or brightness especially noticeable along sharp edges in the image.

Because of this, other demosaicing methods attempt to identify high-contrast edges and only interpolate along these edges, but not across them.




https://en.wikipedia.org/wiki/Color_vision

# Camera Vision (15/33)

Once demosaiced, every pixel is an array of three values: **R, G, B**.

RGB is an additive color model used by screens and sensors and each value is represented as an 8-bit value (depends on the ADC of the sensor).

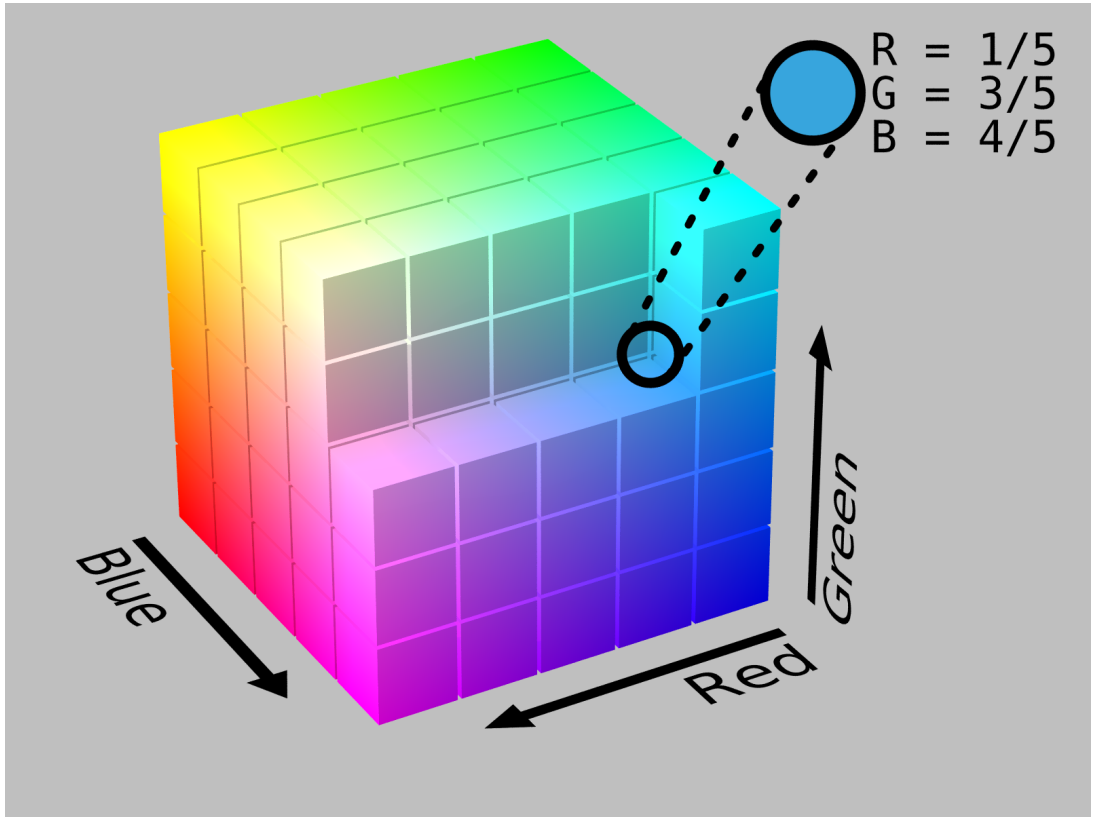
If $R = 255, G = 255, B = 255$, what color do we see?

White

If $R = 255, G = 0, B = 0$, what color do we see?

Red

RGB is represented as a 3D Cartesian cube, where $x, y, z$ correspond to Red, Green, and Blue intensities.



# Camera Vision (16/33)

Imagine we write code to track a red ball:

```
if (R > 200 && G < 50 && B < 50) { track(); }
```

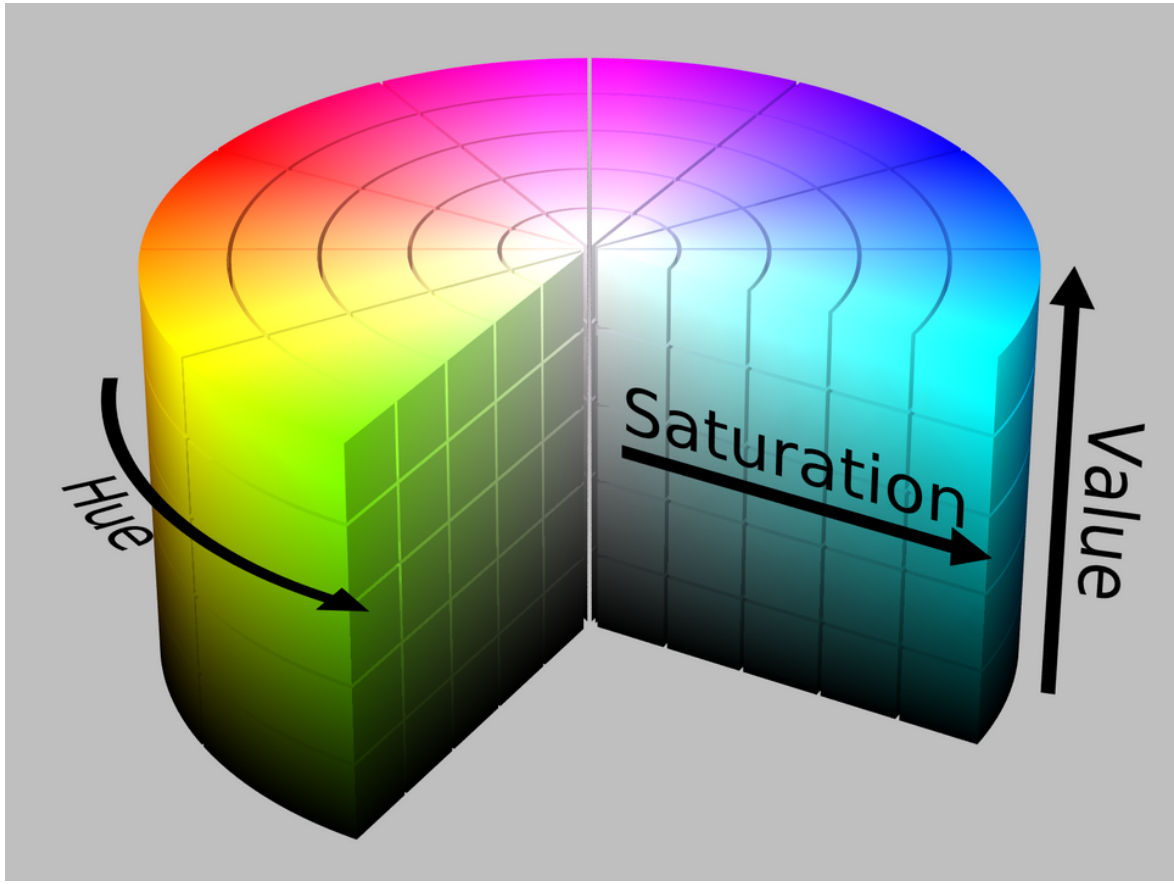
What happens when the robot drives under a shadow?

The total light drops. The pixel might become R=100, G=10, B=10. Our code fails!

In RGB, a change in lighting changes all three variables simultaneously. It couples chrominance (color) with luminance (brightness).

To solve the shadow problem, we transform the RGB cube into a cylinder: The **HSV** color space.

H = **Hue** (The actual color type, e.g., Red, Blue).
S = **Saturation** (How pure the color is, vs. faded white).
V = **Value** (How bright or dark the color is).



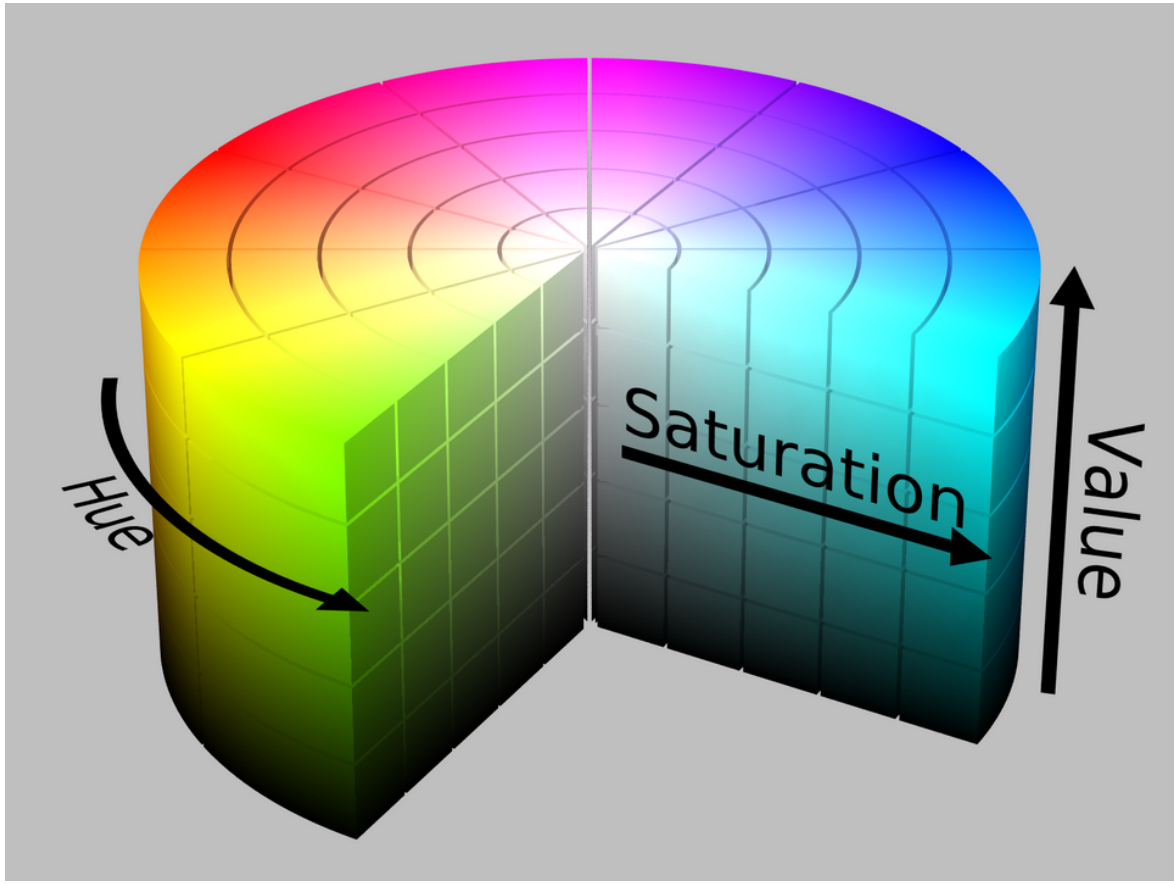

https://en.wikipedia.org/wiki/HSL_and_HSV

# Camera Vision (18/33)

- In the HSV cylinder, Hue is an angle from $0°$ to $360°$.
- $0°$ = Red,
- $120°$ = Green,
- $240°$ = Blue.

Why this is brilliant: If a red ball enters a shadow, its **Value drops**, but its **Hue angle remains "exactly" $0°$!**

This isolates the color from the lighting condition, making our tracking algorithm more robust!




https://en.wikipedia.org/wiki/HSL_and_HSV

# Camera Vision (19/33)

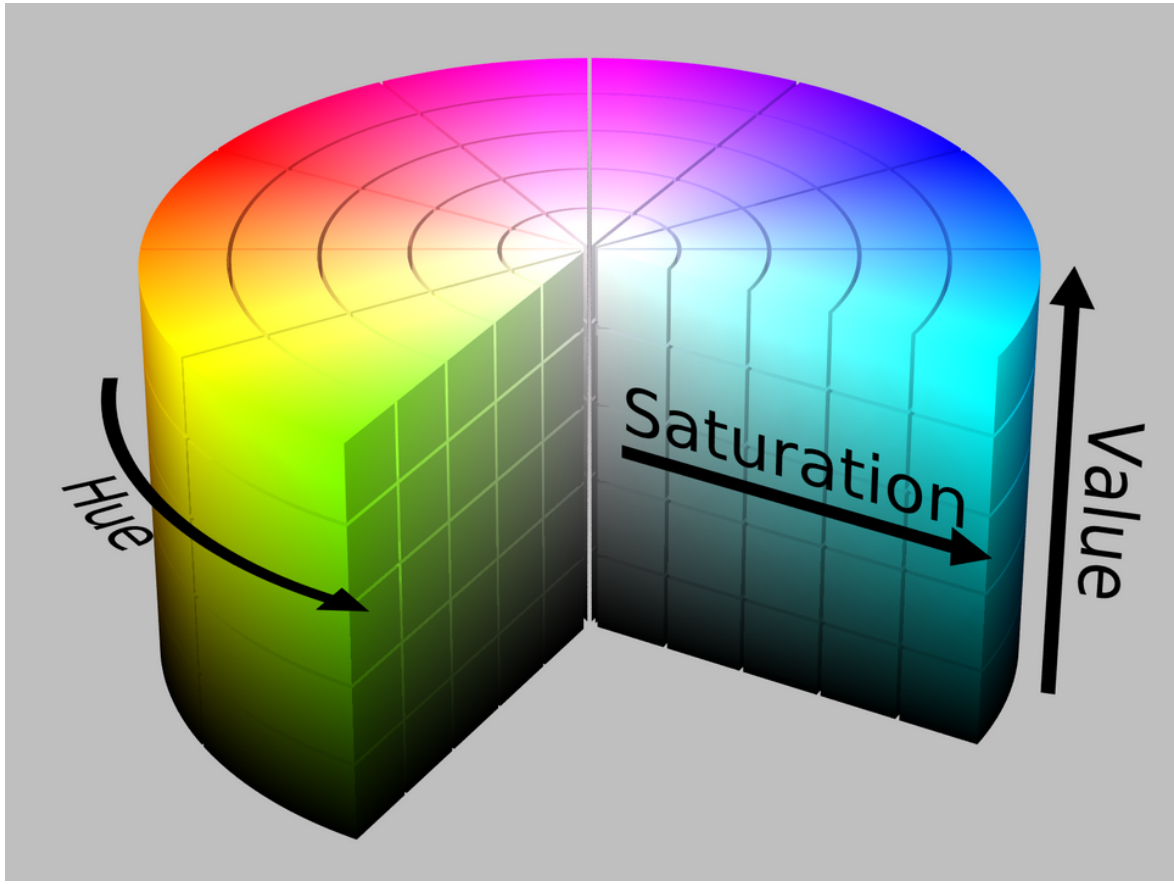
Saturation is the radius (distance from the center axis).

- **S=0%** is pure gray/white (center).
- **S=100%** is a pure, vibrant color (edge).

Value is the height of the cylinder (0 to 100%).

- **V=0%** is pitch black.
- **V=100%** is fully illuminated.

To track an object, we look for pixels that fall within specific ranges.




https://en.wikipedia.org/wiki/HSL_and_HSV

To convert from RGB to HSV the conversion is simple.

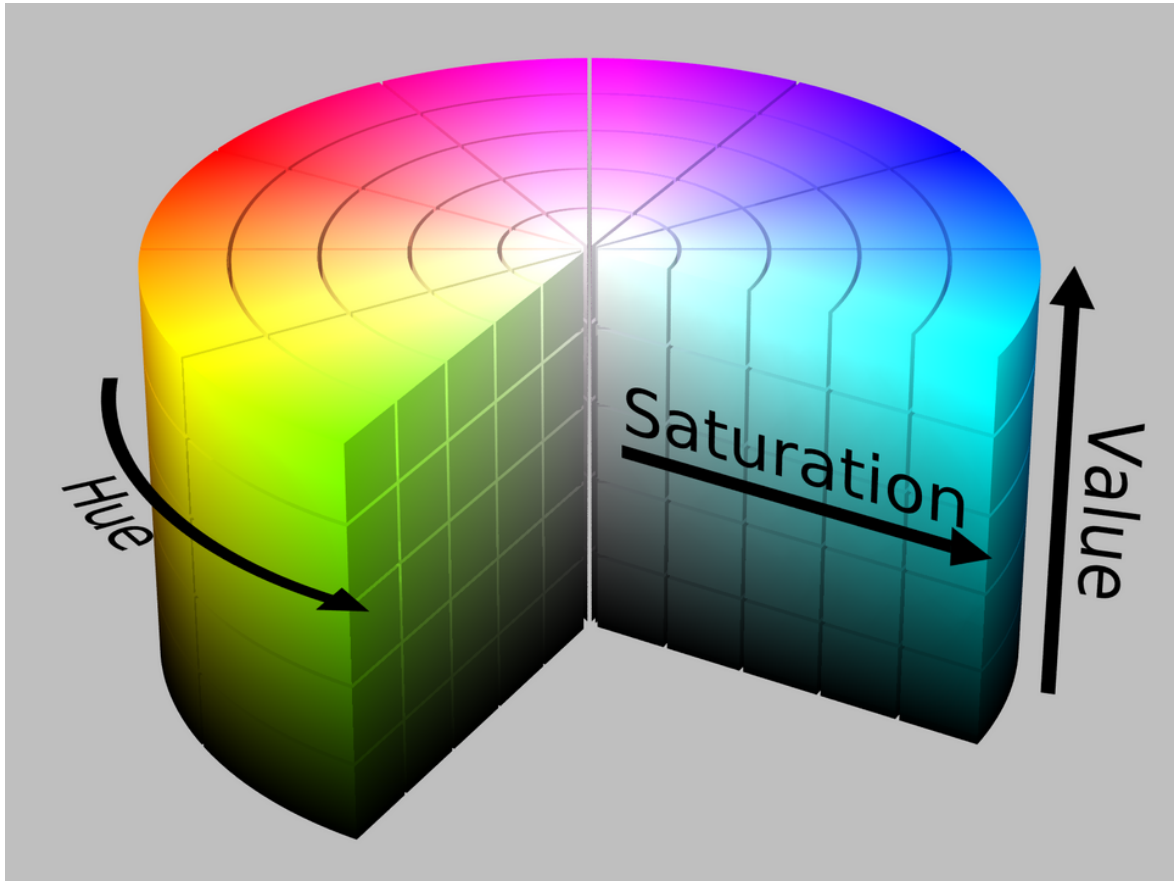
Let $R', G', B'$ be normalized values in $[0,1]$.

$$C_{max} = max(R', G', B'), \qquad C_{min} = min(R', G', B'),$$
$$\Delta = C_{max} - C_{min},$$

$$V = C_{max},$$

$$S = \begin{cases} 0, C_{\max} = 0 \\ \dfrac{\Delta}{C_{\max}} \end{cases},$$

$$\mathrm{H} = 60^\circ \times \begin{cases} \dfrac{\mathrm{G} - \mathrm{B}}{\Delta} \,\%\, 6, C_{max} = \mathrm{R} \\ \dfrac{\mathrm{B} - \mathrm{R}}{\Delta} + 2, C_{max} = \mathrm{G} \\ \dfrac{\mathrm{R} - \mathrm{G}}{\Delta} + 4, C_{max} = \mathrm{B} \end{cases}$$




https://en.wikipedia.org/wiki/HSL_and_HSV

# Camera Vision (21/33)

Which one looks the nicest?



Original Color

https://en.wikipedia.org/wiki/HSL_and_HSV#Disadvantages

HSV value V



CIELAB L*



HSL value L



# Camera Vision (22/33)

While HSL, HSV, and related spaces serve well enough to, for instance, choose a single color, they ignore much of the complexity of color appearance.

Essentially, they trade off **perceptual relevance** for computation speed.

HSL and HSV are simple transformations of RGB which preserve symmetries in the RGB cube unrelated to human perception.

If we plot the RGB gamut in a more **perceptually-uniform space**, such as CIELAB, it becomes immediately clear that the red, green, and blue primaries do not have the same lightness or chroma, or evenly spaced hues.

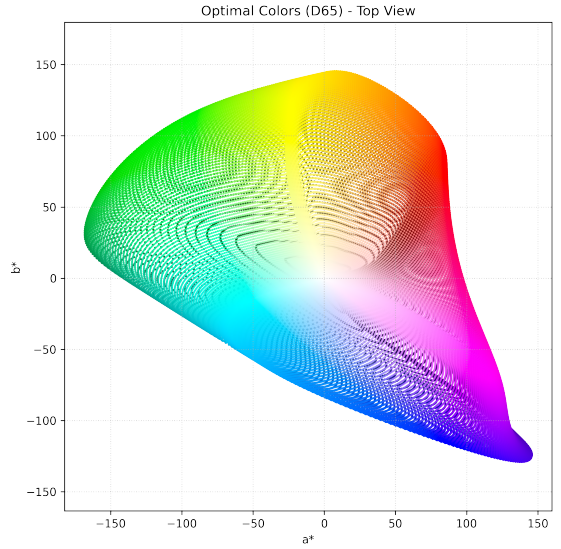
Because HSL and HSV are defined purely with reference to some RGB space, they are not absolute color spaces: to specify a color precisely requires reporting not only HSL or HSV values, but also the characteristics of the RGB space they are based on, including the gamma correction in use.

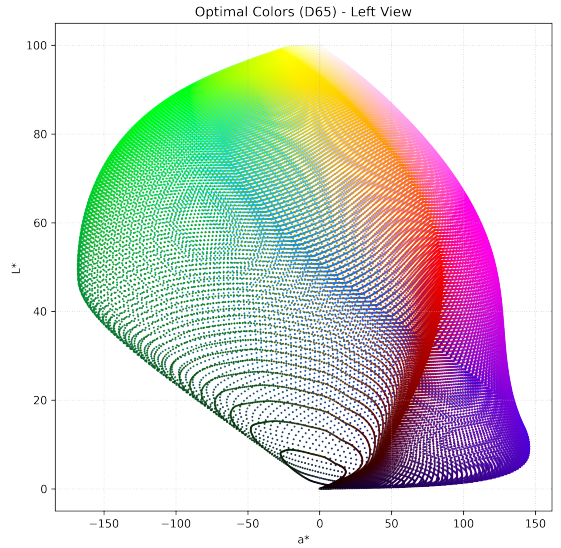
# Camera Vision (23/33)

The **CIELAB** space is three-dimensional and covers the entire gamut (range) of human color perception.

It is based on the opponent model of human vision, where red and green form an opponent pair and blue and yellow form an opponent pair.

The **lightness value**, L* (pronounced "L star"), defines **black** at 0 and **white** at 100.
The **a* axis** is relative to the **green–red** opponent colors, with negative values toward green and positive values toward red.
The **b* axis** represents the **blue–yellow** opponents, with negative numbers toward blue and positive toward yellow.




https://en.wikipedia.org/wiki/CIELAB_color_space

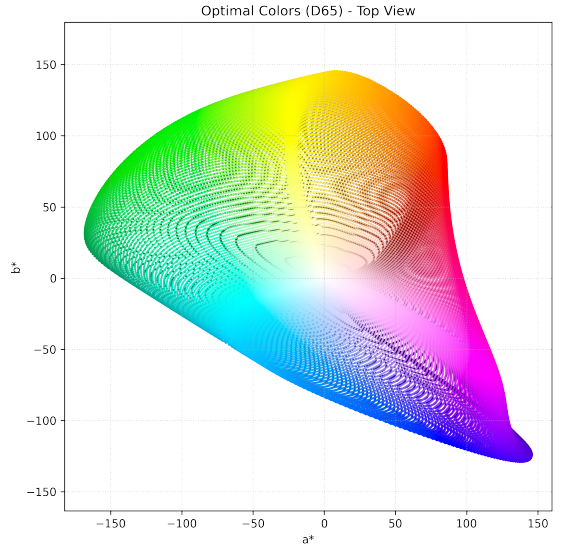
# Camera Vision (24/33)

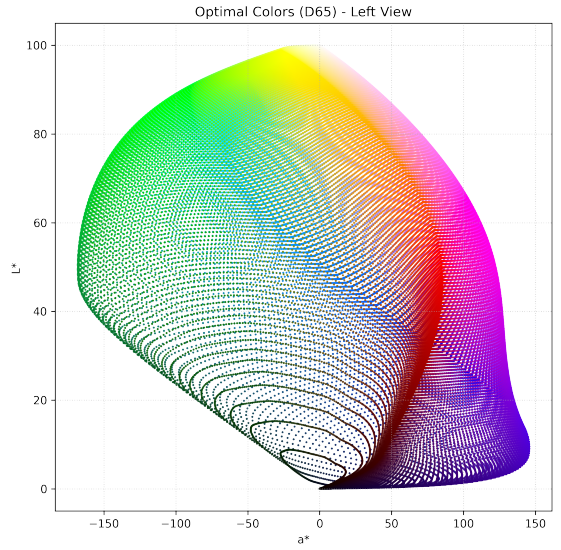
The lightness value, L* in CIELAB is calculated using the cube root of the relative luminance with an offset near black.

This results in an effective power curve with an exponent of approximately 0.43 which represents the human eye's response to light under daylight (photopic) conditions.

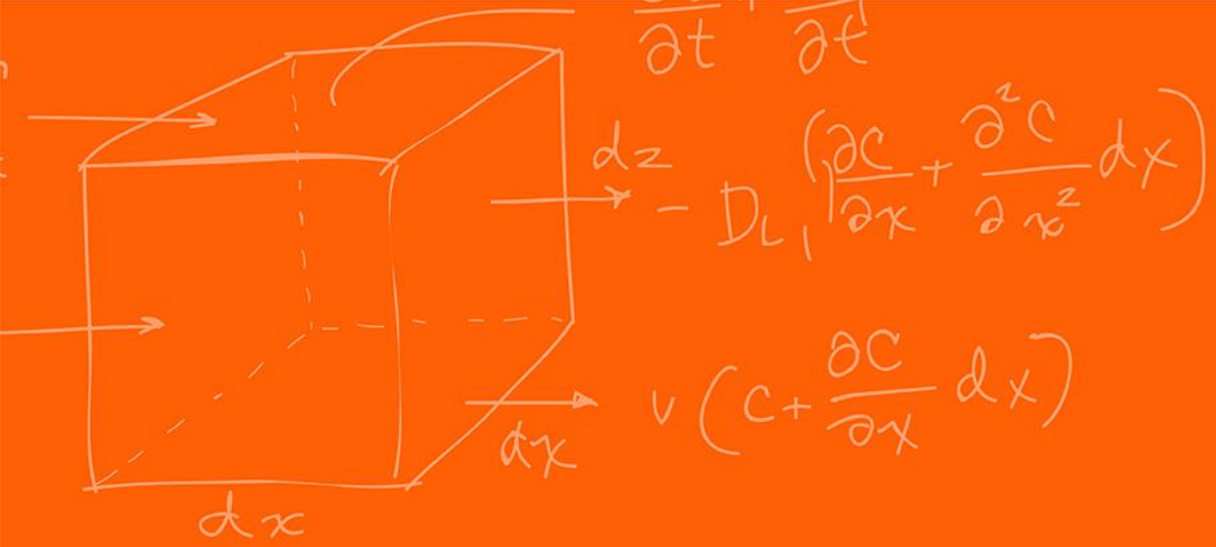
The a* and b* axes are unbounded and depending on the reference white they can easily exceed ±150 to cover the human gamut (range).

Nevertheless, software implementations often clamp these values for practical reasons. For instance, if integer math is being used it is common to clamp a* and b* in the range of –128 to 127.

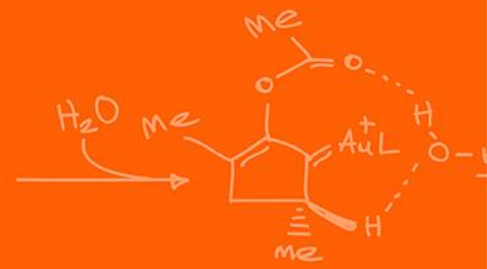



https://en.wikipedia.org/wiki/CIELAB_color_space

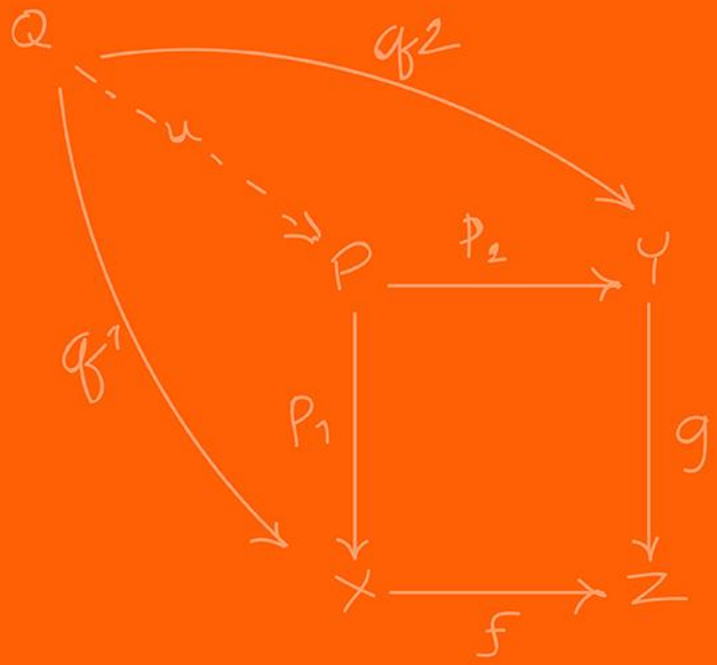




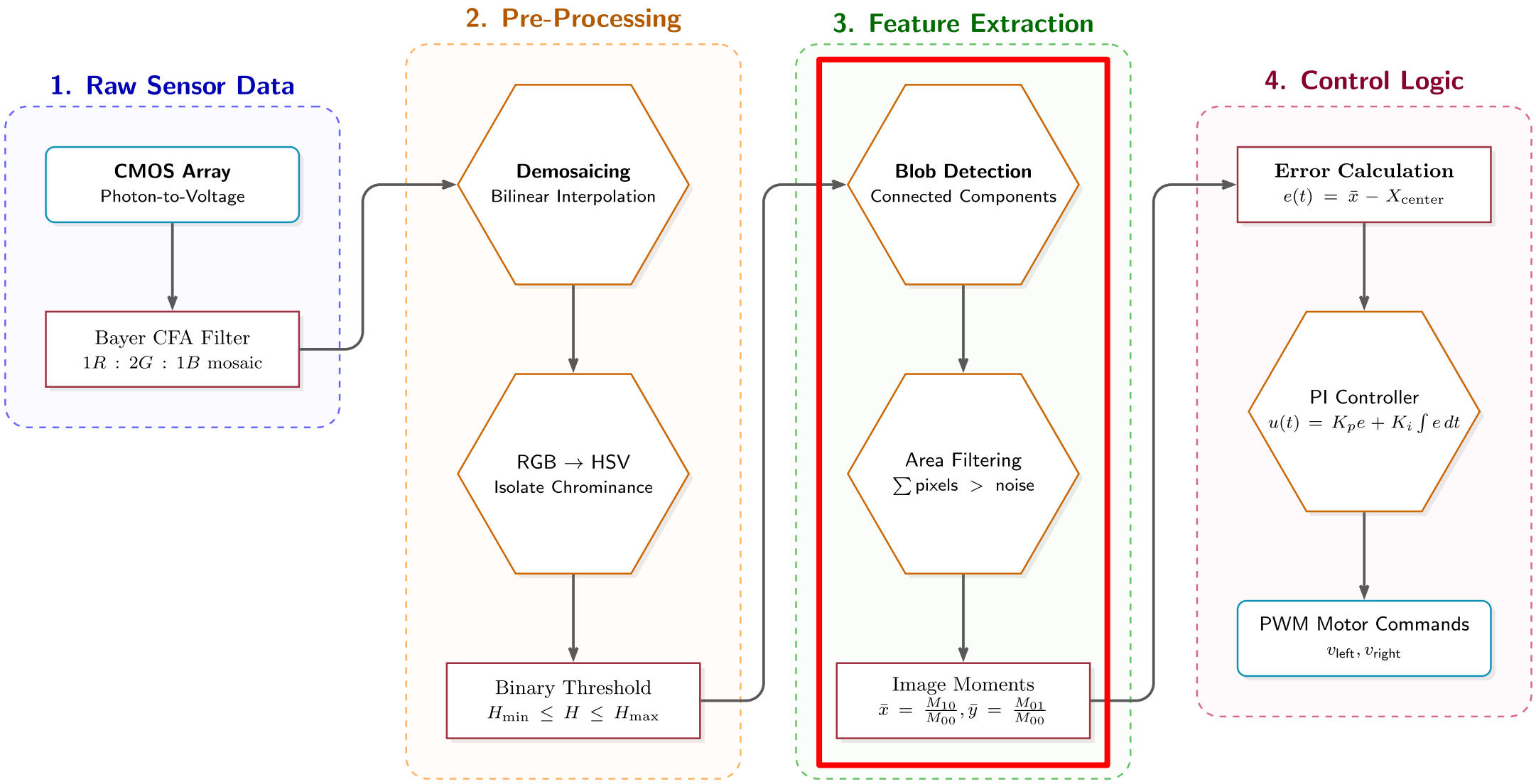




# Blob detection



# Camera Vision (25/33)



# Camera Vision (26/33)

Once we convert our image to HSV, we apply a **Threshold.**

We evaluate at every single pixel: “Does this pixel’s CIELAB fall within my target red range?”

If YES -> Pixel becomes 1 ( Whites)
If NO -> Pixel becomes 0 (Black).

The result is a **Binary Mask**.

We have thrown away all useless data and kept only the target representation.

Does thresholding tell us exactly where the ball is?

# Camera Vision (27/33)

A "Blob" (Binary Large Object) is a contiguous group of connected white pixels in our binary mask.

Blob detection algorithms scan the image to find these groupings and label them as distinct physical objects.

This is known computationally as **Connected Component Labeling (CCL)**.

What simple graph algorithm can you think of could potentially help us?

# Camera Vision (28/33)

The simplest implementation of the algorithm for the Connected Components work is as following:

1. The algorithm scans the image pixel by pixel (raster scan).
2. When it finds a '1', it checks the neighboring pixels (usually 4-connectivity or 8-connectivity).
3. If a neighbor is also a '1', it assigns them the same "Label" (e.g., Object A).
4. It uses a "Union-Find" data structure to merge labels if two separate groups eventually connect.

Otherwise, you can think of this as doing BFS (Breadth-First-Search) on every pixel! (The best algorithms do not do this exactly, but this is the simples way to explain)

# Camera Vision (29/33)

Even with the color filtering, we can still get random glare or noise in the background that will create small fake blobs.

How do we write a rule t ignore these false positives?

Filter blob by **Area** (total pixel count), filter blob by **shape** (the more like a circle the more we want to keep it).

If Blob.Area < MIN_THRESHOLD, we discard it.

We only pass the largest valid blob to the next step.

We have found the blob representing our red ball, now where is it?

We need its (X, Y) coordinate in the camera frame to steer the robot. We do this by calculating the **Centroid** (Center of Mass ) of the blob.

We compute this using **Image Moments**.
In computer vision, a discrete image moment $M_{ij}$ for a binary image $I(x, y)$ is defined as:

$$M_{ij} = \sum_x \sum_y x^i y^j I(x, y)$$

The 0th moment, $M_{00}$, sets $i = 0$ and $j = 0$:

$$M_{00} = \sum_x \sum_y I(x, y)$$

Because $I(x, y)$is either 1 or 0, $M_{00}$ is simply the total sum of white pixels. $M_{00} = \text{Area of the Blob}$.

- To find the center, we need the 1st order moments, $M_{10}$ and $M_{01}$.

$M_{10} = \sum_x \sum_y x \cdot I(x, y)$ (This is the sum of all the X-coordinates of the white pixels).
$M_{01} = \sum_x \sum_y y \cdot I(x, y)$ (This is the sum of all the Y-coordinates of the white pixels).

These act as our weighted spatial totals.

To find the exact center pixel $(\bar{x}, \bar{y})$, we divide the spatial totals by the total area.

$$\bar{x} = \frac{M_{10}}{M_{00}}, \qquad \bar{y} = \frac{M_{01}}{M_{00}}$$

# Camera Vision (32/33)

Now we have $(\bar{x}, \bar{y})$.

Let's say the image is $640 \times 480$ pixels.

The center of the camera is $(320,240)$.

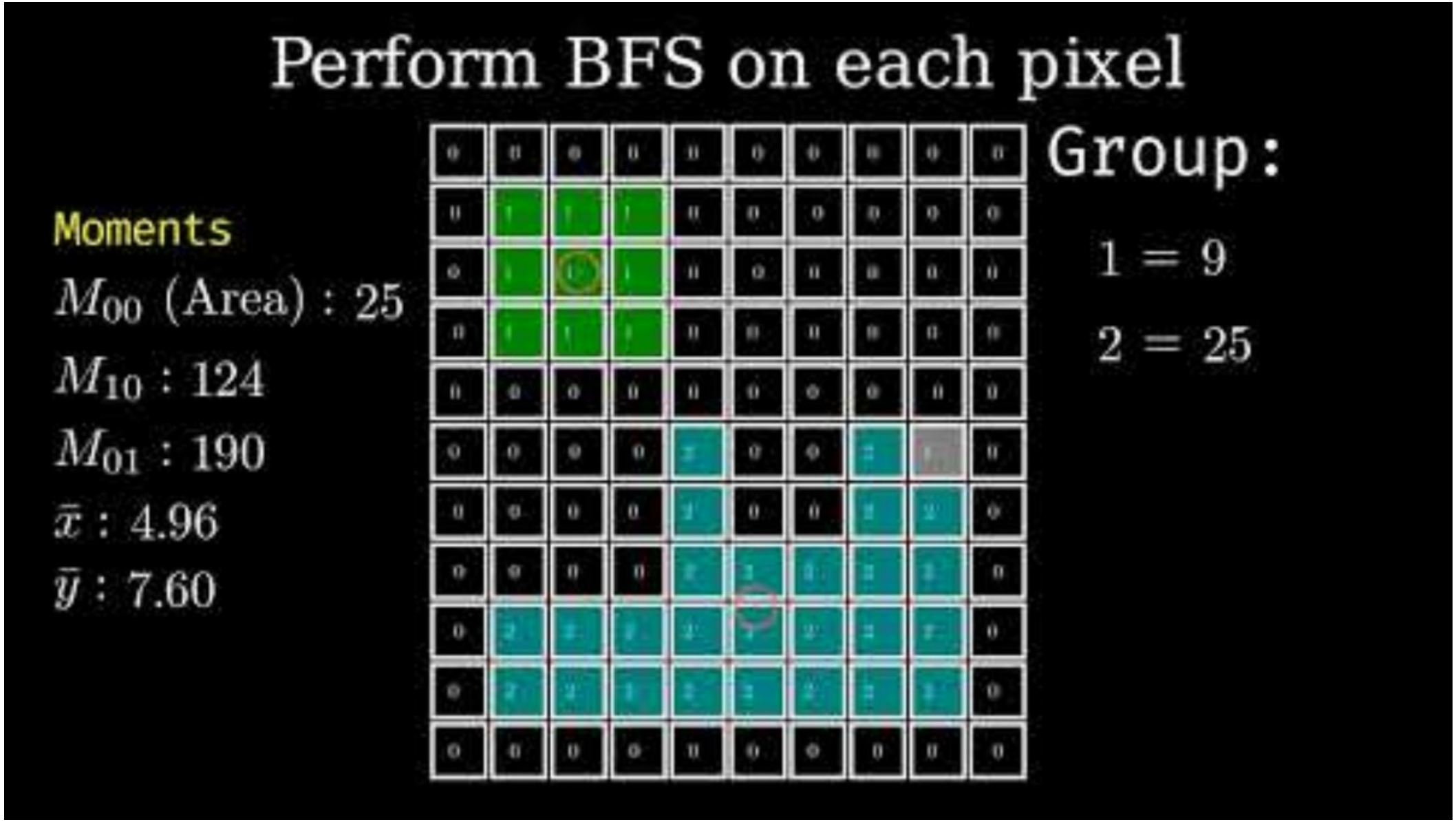
If our centroid is at $\bar{x} = 500$, where is the ball?

It is to the right of the robot.

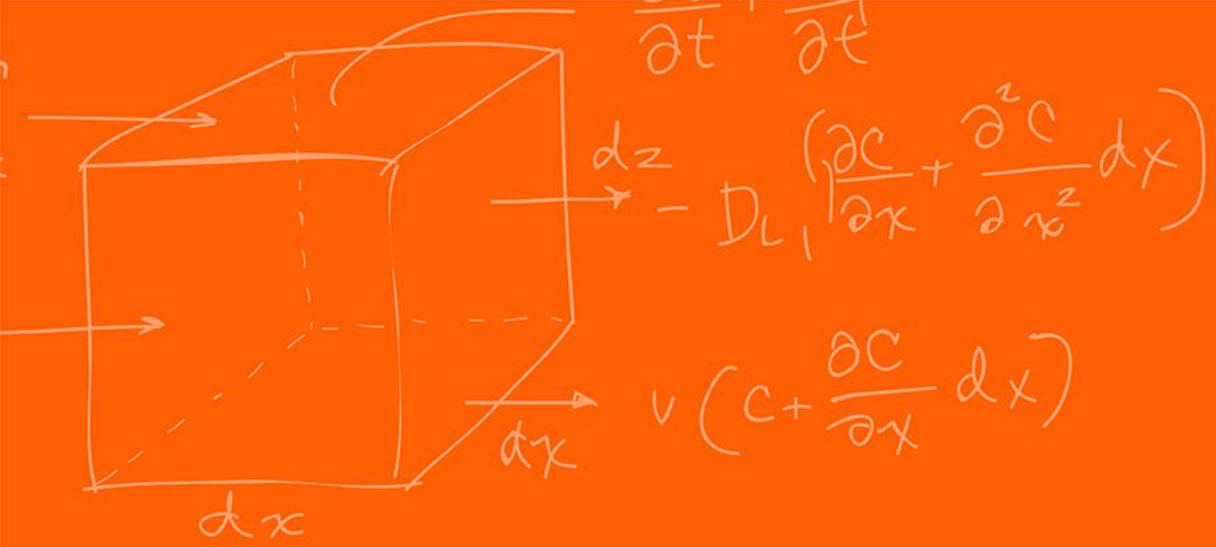
We calculate the error: $e = \bar{x} - X_{center} = 500 - 320 = 180$ pixels.

Given this error, we can pass this as an error to a PI controller to make the robot turn to face the target!

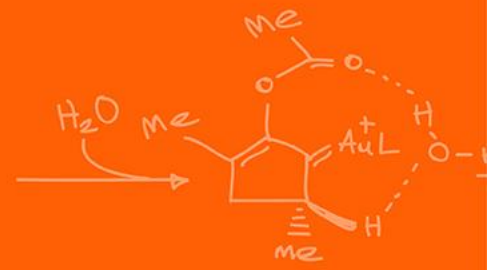
# Camera Vision (33/33)



# Questions? (2/2)









# The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

